Memory Allocation in Linux Device Drivers

A practical guide to allocation choices, DMA, MMIO mappings, and lifetime management.

1) What “memory allocation” means in a driver

Drivers allocate memory for: (a) private state and bookkeeping, (b) I/O buffers, (c) DMA-visible memory, (d) MMIO register mappings, and (e) userspace-visible buffers (mmap/DMA-BUF).

2) The key constraint: can you sleep? (GFP flags & context)

Allocation depends on execution context. In process context you can sleep, so you typically use GFP_KERNEL. In hard IRQ/atomic context you must not sleep, so you use GFP_ATOMIC or preallocated pools.

```
/* Process context allocation */
struct mydev *d = kzalloc(sizeof(*d), GFP_KERNEL);
if (!d)
    return -ENOMEM;

/* Atomic/IRQ context allocation (avoid if possible) */
void my_irq_handler(...) {
    void *p = kmalloc(256, GFP_ATOMIC);
    if (!p) {
        /* handle drop / schedule work / use preallocated buffer */
    }
    ...
    kfree(p);
}
```

3) kmalloc/kzalloc vs slab caches vs vmalloc

kmalloc/kzalloc return physically contiguous memory (best for small/medium objects). For many same-sized objects, create a slab cache (kmem_cache_*). For large buffers without physical contiguity requirements, use vmalloc/vzalloc.

```
/* Slab cache for frequent allocations of same-sized objects */
static struct kmem_cache *node_cache;

int init_cache(void)
{
    node_cache = kmem_cache_create("my_node_cache",
                                   sizeof(struct my_node),
                                   0, SLAB_HWCACHE_ALIGN, NULL);
    return node_cache ? 0 : -ENOMEM;
}

struct my_node *n = kmem_cache_alloc(node_cache, GFP_KERNEL);
if (!n)
    return -ENOMEM;

kmem_cache_free(node_cache, n);

/* Large, non-DMA buffer */
void *big = vzalloc(4 * 1024 * 1024);
if (!big)
    return -ENOMEM;
```

```
vfree(big);
```

4) Page-level allocations

When you need control at page granularity (e.g., rings or SG buffers), allocate pages with alloc_pages() or __get_free_pages().

```
/* Allocate 2^order contiguous pages */
unsigned int order = 2; /* 4 pages */
struct page *pg = alloc_pages(GFP_KERNEL, order);
if (!pg)
    return -ENOMEM;

void *vaddr = page_address(pg);
...
__free_pages(pg, order);
```

5) DMA memory: streaming vs coherent

Use the DMA API so the driver remains portable across cache-coherent and non-coherent systems and works with an IOMMU. For streaming DMA, map/unmap around each transfer. For descriptor rings and control blocks, coherent DMA (dma_alloc_coherent) is common.

```
/* Set DMA mask during probe */
ret = dma_set_mask_and_coherent(dev, DMA_BIT_MASK(64));
if (ret) {
    ret = dma_set_mask_and_coherent(dev, DMA_BIT_MASK(32));
    if (ret)
        return ret;
}

/* Coherent DMA: descriptor ring */
ring = dma_alloc_coherent(dev, ring_bytes, &ring_dma, GFP_KERNEL);
if (!ring)
    return -ENOMEM;

/* Streaming DMA: data buffer */
buf = kmalloc(PKT_SZ, GFP_KERNEL);
if (!buf)
    return -ENOMEM;

dma = dma_map_single(dev, buf, PKT_SZ, DMA_TO_DEVICE);
if (dma_mapping_error(dev, dma)) {
    kfree(buf);
    return -EIO;
}

/* Program device with 'dma' ... */

/* On completion */
dma_unmap_single(dev, dma, PKT_SZ, DMA_TO_DEVICE);
kfree(buf);

/* Free coherent ring at teardown */
dma_free_coherent(dev, ring_bytes, ring, ring_dma);
```

5.1) DMA pools for many small coherent objects

If you need many small coherent chunks (e.g., descriptors), DMA pools reduce fragmentation and simplify allocation.

```
struct dma_pool *pool;
void *desc;
dma_addr_t desc_dma;

pool = dma_pool_create("my_desc_pool", dev, 256, 64, 0);
if (!pool)
    return -ENOMEM;

desc = dma_pool_alloc(pool, GFP_KERNEL, &desc_dma);
if (!desc)
    return -ENOMEM;

/* Use desc + desc_dma */

dma_pool_free(pool, desc, desc_dma);
dma_pool_destroy(pool);
```

6) Device-managed allocations (devm_*)

devm_* helpers automatically release resources when the device unbinds or probe fails, greatly simplifying cleanup.

```
/* Managed driver state */
struct mydev *d = devm_kzalloc(dev, sizeof(*d), GFP_KERNEL);
if (!d)
    return -ENOMEM;

/* Managed MMIO mapping */
res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
base = devm_ioremap_resource(dev, res);
if (IS_ERR(base))
    return PTR_ERR(base);

/* Managed coherent DMA */
d->ring = dmam_alloc_coherent(dev, ring_bytes, &d->ring_dma, GFP_KERNEL);
if (!d->ring)
    return -ENOMEM;
```

7) MMIO mappings (register access)

MMIO space is not normal RAM. Map it and access using readl/writel (or relaxed variants) and appropriate barriers.

```
u32 v;

/* Write register */
writel(cmd, base + REG_CMD);

/* Read register */
v = readl(base + REG_STATUS);

/* Often you also need ordering; consult subsystem guidance */
```

8) Exposing buffers to userspace (mmap)

For coherent DMA memory you can map it into userspace with dma_mmap_coherent(). For other cases, use remap_pfn_range() or vm_insert_page() depending on how the pages are allocated.

```
static int my_mmap(struct file *filp, struct vm_area_struct *vma)
{
    struct mydev *d = filp->private_data;
```

```
    size_t size = vma->vm_end - vma->vm_start;

    if (size > d->ring_bytes)
        return -EINVAL;

    return dma_mmap_coherent(d->dev, vma,
                             d->ring, d->ring_dma,
                             d->ring_bytes);
}
```

9) Typical lifecycle (probe → runtime → remove)

Probe: set DMA mask, allocate state, map registers, allocate rings/buffers, register IRQ/interfaces. Runtime: reuse buffers, map/unmap streaming DMA per transfer. Remove: stop hardware, ensure no DMA in flight, unmap/free buffers.

10) Common pitfalls

1) Using GFP_KERNEL in IRQ context. 2) Trying to DMA from vmalloc memory. 3) Forgetting dma_unmap/dma_sync. 4) Not setting DMA mask. 5) Large kmalloc failures due to fragmentation. 6) Incorrect MMIO access primitives.

11) Quick cheat sheet

Small objects: kmalloc/kzalloc. Many same-sized: slab cache. Huge non-DMA: vmalloc. DMA rings: dma_alloc_coherent. DMA data: pages/kmalloc + dma_map_*. Registers: devm_ioremap_resource.